



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19-03-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

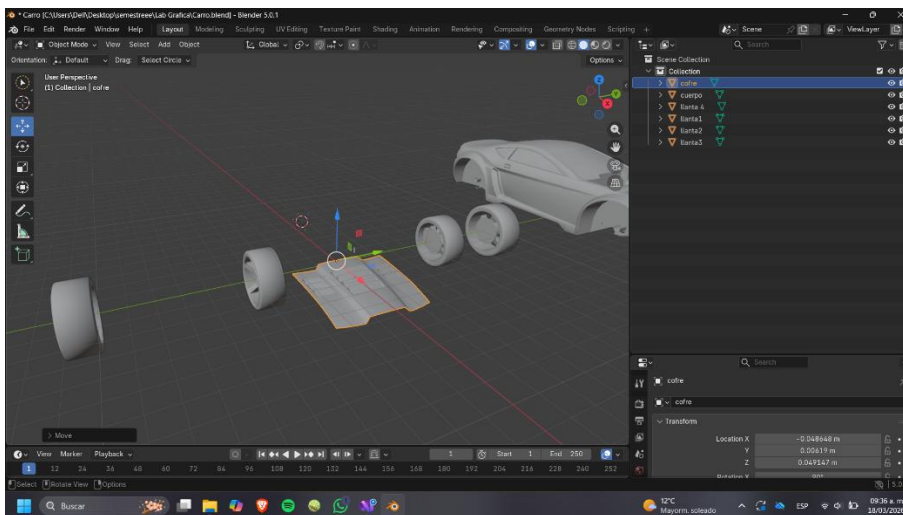
1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada.

El proceso de esta práctica comenzó en blender donde utilizando el modo objeto y el modo edición se separaron individualmente el cuerpo del coche, las 4 llantas, y el cofre respectivamente



Posteriormente se corrigió el origen y la posición de cada parte para que sus ejes coincidieran con el origen del mundo y así más tarde pudieran aplicarse correctamente las rotaciones y traslaciones



Finalmente se exportó cada una de las piezas individualmente en formato .obj

2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.

Para la segunda parte de la práctica se importó el modelo del coche y los demás componentes utilizando la clase Model, la cual permite cargar archivos en formato .obj

```
37 Model Carro_cuerpo;
38 Model Carro_cofre;
39 Model Carro_llanta1;
40 Model Carro_llanta2;
41 Model Carro_llanta3;
42 Model Carro_llanta4;

127 Carro_cuerpo = Model();
128 Carro_cuerpo.LoadModel("Models/cuerpo.obj");
129
130 Carro_cofre = Model();
131 Carro_cofre.LoadModel("Models/cofre.obj");
132
133 Carro_llanta1 = Model();
134 Carro_llanta1.LoadModel("Models/llanta1.obj");
135
136 Carro_llanta2 = Model();
137 Carro_llanta2.LoadModel("Models/llanta2.obj");
138
139 Carro_llanta3 = Model();
140 Carro_llanta3.LoadModel("Models/llanta3.obj");
141
142 Carro_llanta4 = Model();
143 Carro_llanta4.LoadModel("Models/llanta4.obj");
144
```

Para implementar la jerarquía, se utilizó la matriz del cuerpo (modelaux) como base para posicionar cada llanta, cada una se posiciona respecto al cuerpo mediante traslaciones y se les asigna un giro sobre el eje z que se realiza mediante el getarticulacion de cada llanta, ya que el giro de todas las articulaciones individuales se sincronizó al momento de presionar la tecla1 para girar las llantas hacia adelante y la tecla 2 hacia atrás, adicionalmente se le asignó un color diferente a cada llanta para que fuera más fácil identificarlas

```
240 // ===== llanta 1 delantera izq =====
241 model = modelaux;
242 model = glm::translate(model, glm::vec3(4.5f, 1.0f, -3.0f));
243
244 model = glm::rotate(
245     model,
246     mainWindow.getarticulacion1() * toRadians,
247     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
248 );
249
250 color = glm::vec3(0.0f, 1.0f, 0.0f); // verde
251 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
252 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
253 Carro_llanta1.RenderModel();
254
255 // ===== llanta2 trasera izquierda =====
256 model = modelaux;
257 model = glm::translate(model, glm::vec3(-4.5f, 1.0f, -3.0f));
258
259 model = glm::rotate(
260     model,
261     mainWindow.getarticulacion2() * toRadians,
262     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
263 );
264
265 color = glm::vec3(0.0f, 0.0f, 1.0f); // azul
266 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
267 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
268 Carro_llanta2.RenderModel();
269
270
```

```

272 // ===== llanta3 delantera derecha =====
273 model = modelaux;
274 model = glm::translate(model, glm::vec3(4.5f, 1.0f, 3.0f));
275
276 model = glm::rotate(
277     model,
278     mainWindow.getarticulacion3() * toRadians,
279     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
280 );
281
282 color = glm::vec3(1.0f, 1.0f, 0.0f); // amarillo
283 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
284 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
285 Carro_llanta3.RenderModel();
286
287 // ===== llanta4 trasera derecha =====
288 model = modelaux;
289 model = glm::translate(model, glm::vec3(-4.5f, 1.0f, 3.0f));
290
291 model = glm::rotate(
292     model,
293     mainWindow.getarticulacion4() * toRadians,
294     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
295 );
296
297 color = glm::vec3(1.0f, 0.0f, 1.0f); // morado
298 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
299 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
300 Carro_llanta4.RenderModel();
301
302
303 glUseProgram(0);

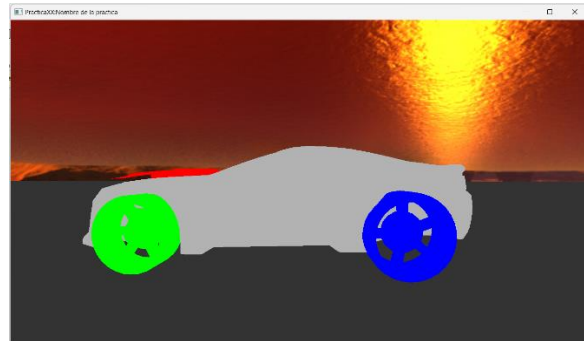
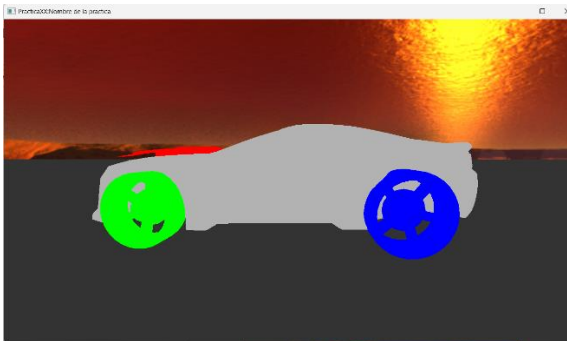
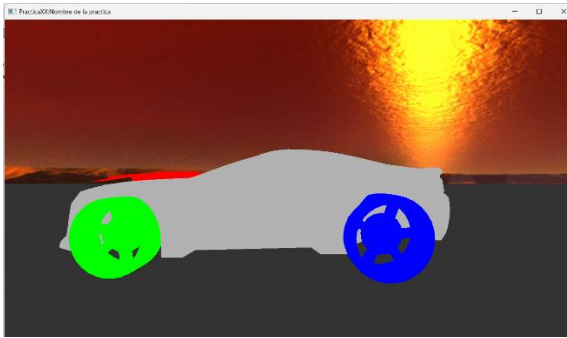
```

Se implementó una sola articulación base (articulacion1) para controlar el movimiento de todas las llantas, replicando su valor en las demás:

```

128 // GIRAR LLANTAS HACIA ADELANTE
129 if (key == GLFW_KEY_1)
130 {
131     theWindow->articulacion1 -= 10.0f;
132
133     // copiar mismo valor a todas las llantas
134     theWindow->articulacion2 = theWindow->articulacion1;
135     theWindow->articulacion3 = theWindow->articulacion1;
136     theWindow->articulacion4 = theWindow->articulacion1;
137 }
138
139 // GIRAR LLANTAS HACIA ATRÁS
140 if (key == GLFW_KEY_2)
141 {
142     theWindow->articulacion1 += 10.0f;
143
144     theWindow->articulacion2 = theWindow->articulacion1;
145     theWindow->articulacion3 = theWindow->articulacion1;
146     theWindow->articulacion4 = theWindow->articulacion1;
147 }

```



3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.

El cofre que también fue cargado como un modelo independiente se integró jerárquicamente al cuerpo usando la misma matriz base (modelaux), este rota sobre el eje Z simulando su apertura y cierre

```

222
223 // ===== cofre =====
224 model = modelaux;
225 model = glm::translate(model, glm::vec3(2.0f, 3.0f, 0.0f));
226
227 model = glm::rotate(
228     model,
229     mainWindow.getarticulacion5() * toRadians,
230     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
231 );
232
233 color = glm::vec3(1.0f, 0.0f, 0.0f); // rojo
234 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
235 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
236 Carro_cofre.RenderModel();
237
238

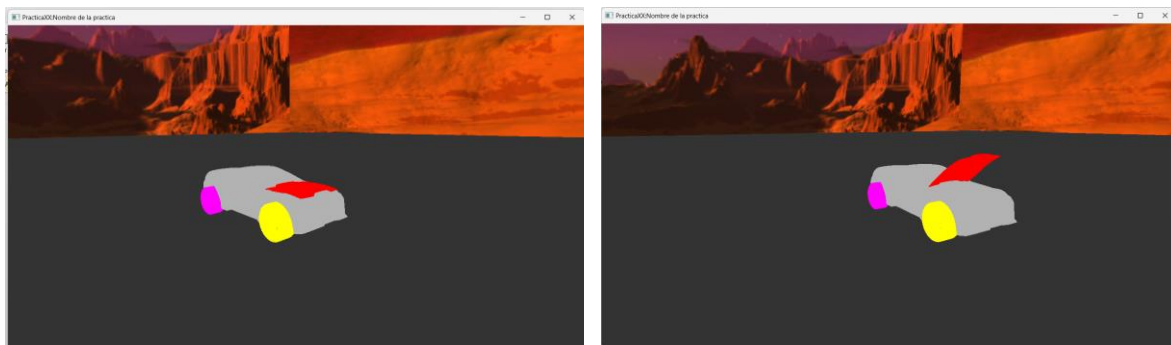
```

Se restringió el movimiento del cofre en un rango de 0° para el cierre para evitar que se hundiera dentro del cuerpo del coche y a 45° la apertura para simular la apertura normal de un cofre de carro, y se asignó la tecla 4 para su apertura y la tecla 5 para el cierre

```

159
160 // ABRIR COFRE
161 if (key == GLFW_KEY_4)
162 {
163     theWindow->articulacion5 += 5.0f;
164
165     if (theWindow->articulacion5 > 45.0f)
166         theWindow->articulacion5 = 45.0f;
167 }
168
169 // CERRAR COFRE
170 if (key == GLFW_KEY_5)
171 {
172     theWindow->articulacion5 -= 5.0f;
173
174     if (theWindow->articulacion5 < 0.0f)
175         theWindow->articulacion5 = 0.0f;
176 }
177

```



4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

Finalmente se implementó el movimiento del coche mediante la variable `muevex` (debido a que el frente y detrás del auto se encuentran posicionados respecto al eje `x`), la cual afecta directamente la transformación del modelo principal, para el avance hacia adelante se designó la tecla `N` y para el avance hacia atrás la tecla `m`, adicionalmente se integró la rotación de las llantas con el movimiento para generar el efecto más realista del movimiento.


```

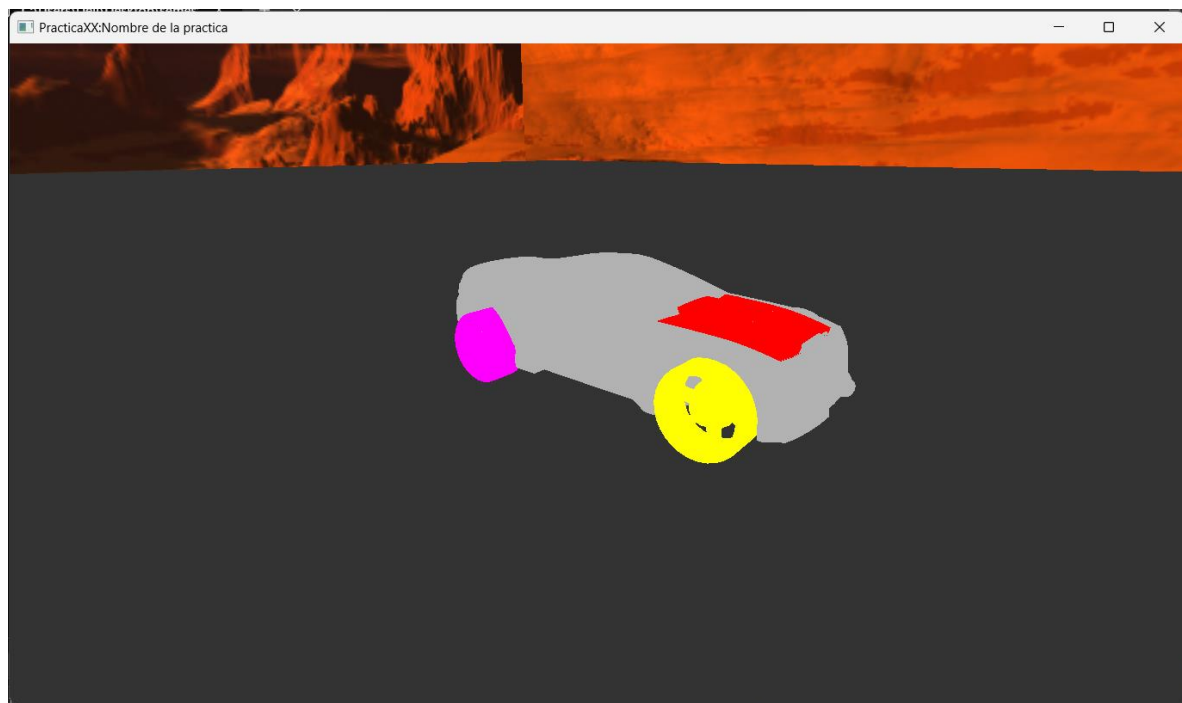
177
178 // AVANZAR
179 if (key == GLFW_KEY_N)
180 {
181     theWindow->muevex += 0.5f;
182
183     // rotación de llantas hacia adelante
184     theWindow->articulacion1 -= 15.0f;
185
186     theWindow->articulacion2 = theWindow->articulacion1;
187     theWindow->articulacion3 = theWindow->articulacion1;
188     theWindow->articulacion4 = theWindow->articulacion1;
189 }
190
191 // RETROCEDER
192 if (key == GLFW_KEY_M)
193 {
194     theWindow->muevex -= 0.5f;
195
196     // rotación de llantas hacia atrás
197     theWindow->articulacion1 += 15.0f;
198
199     theWindow->articulacion2 = theWindow->articulacion1;
200     theWindow->articulacion3 = theWindow->articulacion1;
201     theWindow->articulacion4 = theWindow->articulacion1;
202 }

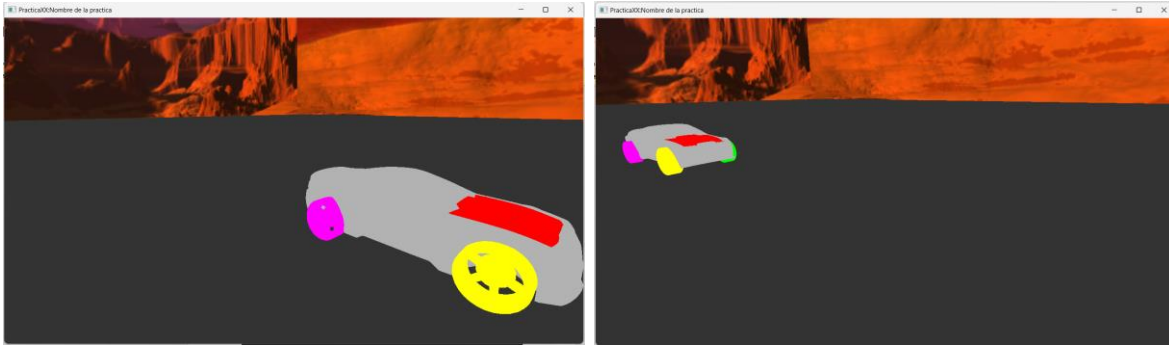
```

```

203 color = glm::vec3(0.7f, 0.7f, 0.7f); //modelo de color gris
204
205 model = glm::mat4(1.0);
206 model = glm::translate(model, glm::vec3(mainWindow.getmuevex(), -1.0f, -1.5f)); //mover padre mover todo
207 //modelaux = model;
208 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
209 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
210 Carro_cuerpo.RenderModel(); //modificar por el modelo sin las 4 llantas ni cofre
211 color = glm::vec3(0.0f, 0.0f, 1.0f);
212
213 modelaux = model;

```





2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No hubo problemas de ejecución en el código únicamente se presentó un error en el control del cofre, ya que inicialmente rotaba en un valores negativos y positivos generando un movimiento poco realista donde se hundía en el cuerpo del coche, para arreglar esto se limitó su rotación a un rango de 0° a 45° , controlando los valores mediante condiciones dentro del manejo de teclado.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Los ejercicios presentaron una complejidad media, ya que implicaron la comprensión y aplicación de conceptos fundamentales de gráficos por computadora como las transformaciones (traslación y rotación) y la jerarquía de modelos.

La parte más importante fue entender cómo funcionan las matrices de transformación y cómo un objeto padre puede afectar a sus hijos. Una vez comprendido este concepto, fue posible implementar correctamente el movimiento conjunto del coche y sus componentes.

El uso de GLM facilitó la implementación de estas transformaciones, aunque requirió un buen entendimiento del orden en que se aplican.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Únicamente se sugiere una explicación más extensa de blender debido que para muchos es la primera vez trabajando con este software por lo que conceptos básicos de su uso son todavía nuevos

c. Conclusión

En esta práctica se logró implementar correctamente un modelo 3D compuesto por múltiples partes, así como su manejo en blender para modificar su posición origen y exportación, aplicando transformaciones jerárquicas y control mediante teclado en c++ y OpenGL.

Se adquirió una mejor comprensión del manejo de matrices en OpenGL, así como de la importancia de la estructura jerárquica en modelos complejos.

Además, se logró integrar interacción en tiempo real, lo que permitió simular el comportamiento de un objeto dinámico, en este caso un coche funcional dentro de un entorno 3D.

4.- Bibliografía en formato APA

LearnOpenGL. (2023). Learn OpenGL – Graphics Programming. Recuperado de: <https://learnopengl.com>

GLFW. (2024). GLFW Documentation. Recuperado de: <https://www.glfw.org/docs/latest/>

OpenGL. (2023). OpenGL Official Documentation. Recuperado de: <https://www.opengl.org/documentation/>

GLM. (2023). OpenGL Mathematics (GLM) Library Documentation. Recuperado de: <https://glm.g-truc.net/>